

Akademia Górniczo-Hutnicza im. Stanisława Staszica w
Krakowie

Projekt realizowany w ramach przedmiotu SDUP

Generator arbitralny AWG na układzie FPGA

DDS, pamięć przebiegu, skalowanie oraz modulacja
sigma-delta

Autorzy: *Kacper Łucki, Hubert Pałka*
Kierunek: *Elektronika i Telekomunikacja*
Prowadzący: *dr. Ernest Jamro*

Kraków, 2026

Streszczenie

W raporcie przedstawiono projekt generatora przebiegów arbitralnych AWG (ang. *Arbitrary Waveform Generator*) zrealizowanego na układzie FPGA Zynq-7010 znajdującym się na płycie Zybo Z7. Generator wykorzystuje 32-bitowy akumulator fazy DDS/NCO, pamięć BRAM przechowującą 4096 próbek 16-bitowych, potokowy układ skalowania amplitudy i dodawania składowej stałej oraz pierwszorzędowy modulator sigma–delta. Fizycznym wyjściem układu jest jednobitowy strumień cyfrowy, którego średnia lokalna odpowiada zadanej wartości próbki. Po odpowiednim uśrednieniu lub filtracji możliwa jest obserwacja odtworzonego przebiegu na oscyloskopie.

Rdzeń sprzętowy został opakowany w autorski blok IP z interfejsem AXI4-Lite. Konfiguracja odbywa się za pomocą czterech rejestrów: sterowania, kroku fazy, wzmocnienia i offsetu oraz portu zapisu pamięci przebiegu. Aktualna, zweryfikowana ścieżka komunikacji wykorzystuje XSDB/JTAG. Aplikacja desktopowa w Pythonie i PyQt6 generuje próbki oraz skrypt TCL, który przy użyciu poleceń `mwr` zapisuje dane bezpośrednio do przestrzeni adresowej AXI. Zrezygnowano z obsługi UART w oprogramowaniu SDK, ponieważ nie była ona używana w działającym torze demonstracyjnym.

Projekt zweryfikowano w symulacji SystemVerilog, przez implementację w Vivado 2018.3 oraz pomiar rzeczywistego sygnału. Dla zegara logiki 100 MHz uzyskano dodatni zapas czasowy WNS równy 2,621 ns i brak naruszeń czasowych. Na oscyloskopie zaobserwowano stabilny przebieg o częstotliwości około 1 kHz.

Spis treści

Streszczenie	i
1 Wprowadzenie	1
1.1 Motywacja	1
1.2 Cel projektu	1
1.3 Zakres raportu	2
2 Architektura systemu	3
2.1 Widok ogólny	3
2.2 Platforma sprzętowa	3
2.3 Podział projektu na katalogi	4
3 Generowanie przebiegu metodą DDS	5
3.1 Akumulator fazy	5
3.2 Rozdzielczość częstotliwości	5
3.3 Pamięć przebiegu	5
4 Skalowanie amplitudy i offsetu	7
4.1 Format współczynnika	7
4.2 Potok obliczeniowy	7
5 Modulator sigma–delta	8
5.1 Zasada działania	8
5.2 Znaczenie filtracji	8
5.3 Ograniczenia rozwiązania jednobitowego	8
6 Interfejs AXI4-Lite	9
6.1 Mapa rejestrów	9
6.2 Obsługa zapisu pamięci	9
7 Sterowanie przez XSDB/JTAG	10
7.1 Powód wyboru rozwiązania	10

7.2	Przykładowa sekwencja	10
8	Aplikacja AWG Signal Forge Qt	12
8.1	Funkcje aplikacji	12
8.2	Generowanie próbek	13
8.3	Walidacja połączenia	13
9	Weryfikacja symulacyjna	14
9.1	Testbench	14
10	Synteza i implementacja	15
10.1	Wykorzystanie zasobów	15
10.2	Analiza czasowa	15
10.3	Ograniczenia XDC	16
11	Pomiary na oscyloskopie	17
11.1	Przebieg sinusoidalny	17
11.2	Przebieg niestandardowy	18
11.3	Interpretacja wyników	19
12	Problemy napotkane podczas realizacji	20
12.1	Długa ścieżka kombinacyjna	20
12.2	Sterowanie z komputera	20
12.3	Aktualizacja całej tablicy	20
12.4	Charakter wyjścia sigma-delta	20
13	Możliwe kierunki rozwoju	21
13.1	Dedykowany tor DAC	21
13.2	Szybsze ładowanie próbek	21
13.3	Dwa banki pamięci	21
13.4	Rozszerzenie funkcji generatora	21
14	Podsumowanie	22
A	Aktualny minimalny firmware SDK	23
B	Procedura uruchomienia	25

Spis rysunków

2.1	Uproszczony przepływ danych w aktualnej wersji systemu.	3
8.1	Aplikacja AWG Signal Forge Qt z wybranym trybem XSDB/JTAG. . . .	12
11.1	Przebieg sinusoidalny o częstotliwości około 1 kHz.	18
11.2	Przykład niestandardowego przebiegu uzyskanego z pamięci próbek. . . .	19

Spis tabel

2.1	Najważniejsze sygnały zewnętrzne.	3
6.1	Mapa rejestrów generatora AWG.	9
10.1	Wykorzystanie wybranych zasobów po rozmieszczeniu.	15

Rozdział 1

Wprowadzenie

1.1 Motywacja

Generatory przebiegów są podstawowymi przyrządami wykorzystywanymi podczas uruchamiania i testowania układów elektronicznych. Typowy generator laboratoryjny udostępnia kilka standardowych przebiegów, takich jak sinusoida, prostokąt, trójkąt lub piła. Generator arbitralny rozszerza tę funkcjonalność, ponieważ pozwala użytkownikowi zdefiniować kształt jednego okresu w postaci tablicy próbek. Ten sam sprzęt może dzięki temu generować przebiegi klasyczne, sumy harmoniczných, sygnały modulowane, impulsy testowe oraz niestandardowe funkcje zadane przez użytkownika.

Układ FPGA jest dobrym środowiskiem do realizacji takiego generatora. Zapewnia deterministyczny zegar, wbudowane pamięci blokowe, szybkie mnożniki DSP oraz możliwość utworzenia własnego interfejsu sterującego. Jednocześnie płytką Zybo Z7 nie zawiera klasycznego wielobitowego przetwornika cyfrowo-analogowego podłączonego bezpośrednio do logiki programowalnej. W projekcie wykorzystano więc jednobitową modulację sigma–delta, która zamienia słowo próbki na szybki strumień logiczny. Rozwiązanie jest proste sprzętowo, a po filtracji pozwala uzyskać sygnał analogowy odpowiedni do demonstracji zasady działania AWG.

1.2 Cel projektu

Głównym celem było opracowanie kompletnego toru generacji przebiegu, obejmującego:

1. przygotowanie tablicy 4096 próbek 16-bitowych,
2. odczyt próbek z częstotliwością zależną od zadanego kroku fazy,
3. zmianę amplitudy i offsetu bez modyfikowania zawartości pamięci,
4. konwersję próbki wielobitowej na jednobitowy strumień sigma–delta,
5. udostępnienie konfiguracji przez interfejs AXI4-Lite,
6. przesyłanie danych z komputera przez XSDB/JTAG,
7. przygotowanie aplikacji użytkownika w PyQt6,

8. weryfikację symulacyjną, implementacyjną i pomiarową.

Założono, że rozwiązanie ma być możliwie proste do demonstracji. Nie jest to generator pomiarowy o gwarantowanych parametrach analogowych. Projekt pokazuje natomiast kompletny przepływ danych od aplikacji na komputerze, przez magistralę AXI i logikę FPGA, aż do fizycznego wyjścia mierzonego oscyloskopem.

1.3 Zakres raportu

Raport opisuje aktualną wersję projektu. Szczególną uwagę poświęcono blokom HDL, mapie rejestrów, sposobowi generowania częstotliwości, implementacji potoku skalującego oraz sterowaniu przez XSDB. W osobnym rozdziale przedstawiono rezygnację z komunikacji UART w SDK. W końcowej części omówiono wyniki syntezy, implementacji, symulacji i pomiarów.

Rozdział 2

Architektura systemu

2.1 Widok ogólny

System można podzielić na warstwę aplikacji komputerowej, warstwę sterowania i warstwę generacji sprzętowej. Aplikacja PyQt6 tworzy tablicę próbek oraz parametry generatora. Dane są zamieniane na skrypt XSDB/TCL. Narzędzie XSDB łączy się przez JTAG z systemem MicroBlaze i wykonuje bezpośrednie zapisy do rejestrów bloku AWG. Interfejs AXI4-Lite przekazuje wartości do rdzenia, a rdzeń generuje sygnał `sd_out`.



Rysunek 2.1: Uproszczony przepływ danych w aktualnej wersji systemu.

2.2 Platforma sprzętowa

Projekt zrealizowano dla układu `xc7z010c1g400-1`. Zewnętrzny zegar 125 MHz jest doprowadzony do wejścia K17. Blok Clocking Wizard wytwarza zegar 100 MHz używany przez MicroBlaze, magistralę AXI i rdzeń AWG. Wyjście sigma-delta wyprowadzono na Pmod JA1, pin N15. Przyciski BTN0 i BTN1 pełnią funkcję wejść resetujących. Dla sygnałów asynchronicznych i wyjścia sigma-delta zdefiniowano odpowiednie ścieżki fałszywe w pliku ograniczeń XDC.

Tabela 2.1: Najważniejsze sygnały zewnętrzne.

Sygnał	Pin	Standard	Funkcja
<code>clk_125MHz</code>	K17	LVC MOS33	wejściowy zegar systemowy 125 MHz
<code>reset_rtl_0</code>	K18	LVC MOS33	reset z BTN0
<code>reset_rtl_0_0</code>	P16	LVC MOS33	drugi reset z BTN1
<code>sd_out_0</code>	N15	LVC MOS33	jednobitowe wyjście AWG, Pmod JA1

2.3 Podział projektu na katalogi

Kod został rozdzielony na część demonstracyjną rdzenia, projekt IP oraz pełny system Vivado. Najważniejsze katalogi przedstawiono poniżej.

- `awg_sv_project/rtl` – moduły SystemVerilog rdzenia AWG,
- `awg_sv_project/tb` – testbench, pamięć testowa i narzędzia analizy,
- `GeneratorArbitralny/ip_repo/awg_axi_1.0` – własny blok IP AXI,
- `gen_arb_z_ip` – pełny projekt Vivado z MicroBlaze i blokami AXI,
- `gen_arb_z_ip.sdk/awg_gen` – minimalna aplikacja startowa SDK,
- `tools/awg_qt` – aplikacja desktopowa AWG Signal Forge Qt.

Rozdział 3

Generowanie przebiegu metodą DDS

3.1 Akumulator fazy

Podstawą adresowania pamięci przebiegu jest 32-bitowy akumulator fazy. W każdym aktywnym takcie zegara do rejestru fazy dodawana jest wartość `phase_step`. Najstarsze 12 bitów akumulatora tworzy adres pamięci BRAM. Ponieważ pamięć ma $2^{12} = 4096$ komórek, cały zakres akumulatora odpowiada jednemu okresowi przebiegu.

Zależność między żadaną częstotliwością wyjściową, częstotliwością zegara i krokiem fazy wynosi:

$$\text{phase_step} = \text{round} \left(\frac{f_{out}}{f_{clk}} 2^{32} \right). \quad (3.1)$$

Dla $f_{clk} = 100$ MHz i $f_{out} = 1$ kHz otrzymuje się wartość zbliżoną do 0x0000A7C6. Jest to wartość używana podczas startu firmware oraz wyświetlana w aplikacji.

3.2 Rozdzielczość częstotliwości

Minimalna zmiana częstotliwości wynikająca z jednostkowej zmiany kroku fazy to:

$$\Delta f = \frac{f_{clk}}{2^{32}}. \quad (3.2)$$

Dla zegara 100 MHz daje to około 0,0233 Hz. Oznacza to bardzo dobrą rozdzielczość strojenia numerycznego. Rzeczywista jakość przebiegu zależy jednak również od długości tablicy, filtracji wyjścia, częstotliwości zegara sigma-delta i jakości toru analogowego.

3.3 Pamięć przebiegu

Moduł `wave_bram` opisuje synchroniczną pamięć 4096 słów po 16 bitów. Pamięć może zostać zainicjalizowana z pliku `waveform.mem`, dzięki czemu generator działa bezpośrednio po konfiguracji FPGA. Jednocześnie posiada port zapisu, który pozwala aplikacji zastąpić próbki podczas pracy.

Każdy zapis przez rejestr `WAVE_WRITE` zawiera adres próbki w bitach $[27 : 16]$ i wartość próbki w bitach $[15 : 0]$. Dla adresu a i próbki s słowo zapisywane do AXI ma postać:

$$W = ((a \& 0xFFF) \ll 16) \mid (s \& 0xFFFF). \quad (3.3)$$

Rozdział 4

Skalowanie amplitudy i offsetu

4.1 Format współczynnika

Amplituda jest ustawiana za pomocą 16-bitowego współczynnika `amplitude_q16`. Wartość `0xFFFF` odpowiada wzmocnieniu bliskiemu jedności, a mniejsze wartości zmniejszają amplitudę. Wynik mnożenia próbki przez współczynnik jest przesuwany o 16 bitów w prawo. Następnie dodawany jest 16-bitowy offset.

W uproszczeniu operację można zapisać jako:

$$y[n] = \text{sat}_{16} \left(\left\lfloor \frac{x[n]G}{2^{16}} \right\rfloor + O \right), \quad (4.1)$$

gdzie $x[n]$ oznacza próbkę z BRAM, G jest współczynnikiem Q0.16, O jest offsetem, a sat_{16} ogranicza wynik do zakresu $0 \dots 65535$.

4.2 Potok obliczeniowy

Pierwsza wersja układu wykonywała odczyt BRAM, mnożenie, dodanie offsetu, saturację i wejście do sigma–delta w jednej długiej ścieżce kombinacyjnej. Powodowało to problemy z timingiem. Blok `scaler` został więc podzielony na etapy rejestrowe:

1. rejestracja próbki i parametrów konfiguracyjnych,
2. rejestracja wyniku mnożenia,
3. wybór wartości po skalowaniu, dodanie offsetu i saturacja.

Opóźnienie potoku wzrosło o kilka taktów, lecz dla sygnału okresowego nie zmienia to częstotliwości ani kształtu ustalonego przebiegu. Zaletą jest krótsza ścieżka krytyczna i dodatni zapas czasowy po implementacji.

Rozdział 5

Modulator sigma–delta

5.1 Zasada działania

Modulator sigma–delta pierwszego rzędu tworzy jednobitowy strumień, którego gęstość stanów wysokich odpowiada poziomowi próbki wejściowej. W implementacji wykorzystano akumulator 20-bitowy. Próbką 16-bitową jest rozszerzana przez dopisanie czterech zer i dodawana do akumulatora. Bit przeniesienia sumy staje się wartością `sd_out`, natomiast młodsze bity pozostają stanem akumulatora na następny takt.

Można to opisać zależnościami:

$$A[n + 1] = (A[n] + 2^4 x[n]) \bmod 2^{20}, \quad (5.1)$$

$$b[n] = \left\lfloor \frac{A[n] + 2^4 x[n]}{2^{20}} \right\rfloor. \quad (5.2)$$

Dla próbki równej zero wyjście jest wymuszane na zero, a dla maksymalnej próbki na jeden. Specjalne przypadki eliminują niepożądane zachowanie na granicach zakresu.

5.2 Znaczenie filtracji

Sygnal `sd_out` nie jest bezpośrednio wielopoziomowym napięciem analogowym. Jest szybkim przebiegiem logicznym. Wartość analogowa jest zawarta w jego średniej. Do praktycznego odtworzenia sygnału można użyć filtra RC lub innego filtra dolnoprzepustowego. Oscyloskop i pasmo badanego toru również powodują częściowe uśrednienie. Parametry filtra muszą stanowić kompromis między tłumieniem składowej przełączającej a zachowaniem użytecznego pasma generatora.

5.3 Ograniczenia rozwiązania jednobitowego

Najważniejsze ograniczenia to szum kwantyzacji, zależność jakości od nadpróbkowania, wrażliwość na filtr wyjściowy i ograniczona amplituda napięciowa wynikająca ze standardu I/O FPGA. Rozwiązanie jest wystarczające do projektu demonstracyjnego, lecz nie zastępuje precyzyjnego przetwornika DAC w generatorze laboratoryjnym.

Rozdział 6

Interfejs AXI4-Lite

6.1 Mapa rejestrów

Własny blok `awg_axi` udostępnia cztery rejestry 32-bitowe w przestrzeni adresowej rozpoczynającej się od `0x44A00000`. Mapa jest celowo niewielka, aby sterowanie było proste zarówno z programu C, jak i z XSDB.

Tabela 6.1: Mapa rejestrów generatora AWG.

Offset	Nazwa	Znaczenie
0x00	CONTROL	bit 0: enable, bit 1: reset rdzenia
0x04	PHASE_STEP	32-bitowy krok akumulatora fazy DDS
0x08	GAIN_OFFSET	gain Q0.16 w [15 : 0], offset w [31 : 16]
0x0C	WAVE_WRITE	adres próbki w [27 : 16], dane w [15 : 0]

6.2 Obsługa zapisu pamięci

Zapis do `WAVE_WRITE` generuje jednocyklowy impuls `wave_wr_en`. Adres i dane są rejestrowane, a następnie przekazywane do portu zapisu `wave_bram`. Odczyt pamięci przez DDS odbywa się równolegle. Takie rozwiązanie pozwala aktualizować przebieg bez tworzenia osobnego kontrolera pamięci.

Podczas ładowania całej tablicy bezpieczną procedurą jest zatrzymanie lub reset generatora, zapis wszystkich 4096 komórek, ustawienie parametrów i ponowne włączenie rdzenia. Skrypt generowany przez aplikację wykonuje te kroki w ustalonej kolejności.

Rozdział 7

Sterowanie przez XSDB/JTAG

7.1 Powód wyboru rozwiązania

W trakcie prac rozważano komunikację szeregową przez AXI UARTLite, jednak tor ten nie był niezawodnym elementem demonstracji. Ostatecznie w działającej wersji wykorzystano XSDB/JTAG. To samo połączenie kablowe służy do programowania FPGA i debugowania MicroBlaze, dlatego nie jest wymagany dodatkowy adapter USB-UART.

XSDB umożliwia wybór procesora MicroBlaze i bezpośredni dostęp do przestrzeni adresowej systemu. Polecenie `mwr` zapisuje wskazane słowo 32-bitowe, a `mrd` służy do odczytu kontrolnego. Aplikacja PyQt6 tworzy tymczasowy skrypt TCL i uruchamia `xsdb.bat` lub `xsct.bat` jako osobny proces.

7.2 Przykładowa sekwencja

Listing 7.1: Uproszczony skrypt sterujący XSDB.

```
1 catch {connect -url tcp:127.0.0.1:3121}
2 targets -set -nocase -filter {name =~ "microblaze*#0"} -index 1
3 configparams force-mem-access 1
4
5 # Reset i zatrzymanie rdzenia
6 mwr 0x44A00000 0x00000002
7 mwr 0x44A00000 0x00000000
8
9 # Przykładowy zapis probki: adres 5, wartosc 32768
10 mwr 0x44A0000C 0x00058000
11
12 # Konfiguracja czestotliwosci i gain/offset
13 mwr 0x44A00004 0x0000A7C6
14 mwr 0x44A00008 0x0000FFFF
15
16 # Uruchomienie generatora
17 mwr 0x44A00000 0x00000001
18 configparams force-mem-access 0
19 disconnect
```


Pełna aktualizacja fali wymaga 4096 zapisów do adresu 0x44A0000C. Nie jest to transmisja czasu rzeczywistego, lecz dla laboratoryjnego ustawiania przebiegu jest wystarczająca i prosta do zweryfikowania.

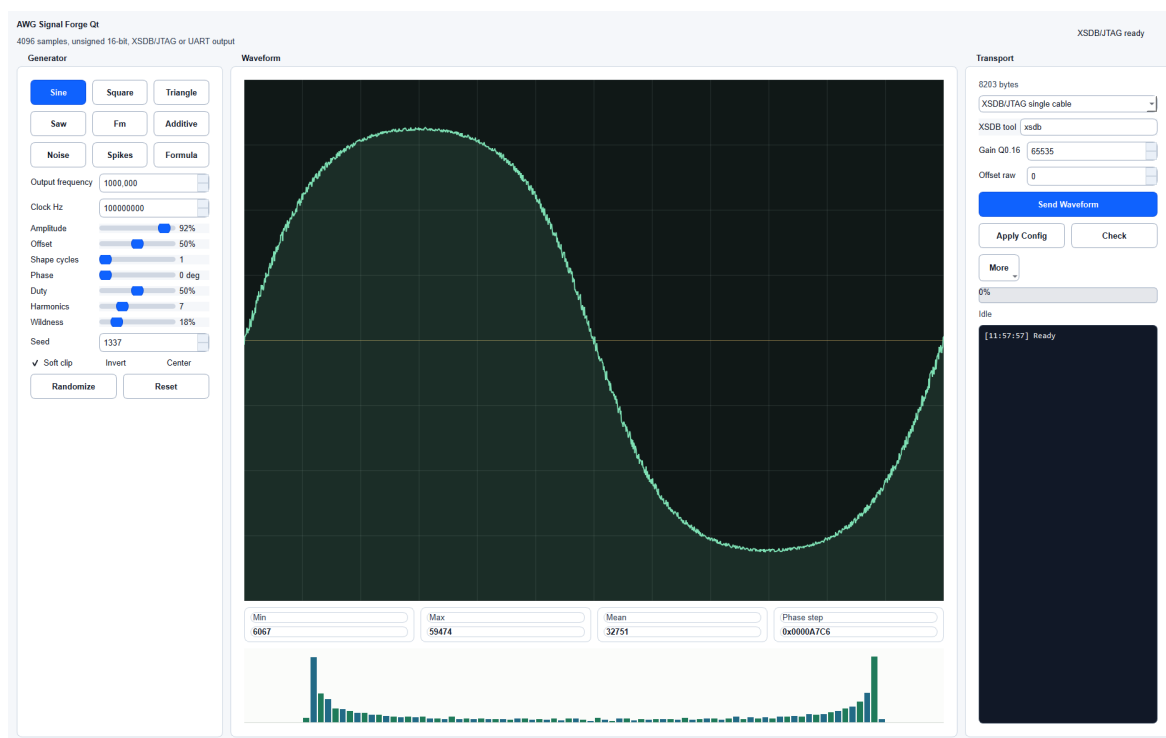
Rozdział 8

Aplikacja AWG Signal Forge Qt

8.1 Funkcje aplikacji

Aplikacja została napisana w Pythonie z wykorzystaniem biblioteki PyQt6. Generuje 4096-elementową tablicę liczb bez znaku z zakresu 0–65535. Dostępne są przebiegi sinusoidalny, prostokątny, trójkątny, piłokształtny, FM, suma harmoniczných, szum, impulsy oraz wzór wpisywany przez użytkownika. Parametry obejmują między innymi liczbę cykli w tablicy, fazę, wypełnienie, liczbę harmoniczných, poziom losowości, odwrócenie i usuwanie składowej stałej.

Na panelu transportu użytkownik podaje ścieżkę do XSDB/XSCT, ustawia gain i offset, a następnie może sprawdzić połączenie, zastosować samą konfigurację lub przesłać całą tablicę. Operacje są wykonywane w osobnym wątku, dzięki czemu interfejs nie blokuje się podczas kilku tysięcy zapisów.



Rysunek 8.1: Aplikacja AWG Signal Forge Qt z wybranym trybem XSDB/JTAG.

8.2 Generowanie próbek

Próbki są najpierw obliczane w zakresie umownym $[-1, 1]$. Następnie stosowane są modyfikacje, takie jak soft clipping, odwrócenie i blokada składowej stałej. Na końcu sygnał jest mapowany do zakresu 16-bitowego:

$$s[n] = \text{round} (65535 \cdot \text{clip} (0.5 + 0.5Ax[n] + O, 0, 1)) . \quad (8.1)$$

Aplikacja pokazuje podgląd przebiegu, histogram wartości, minimum, maksimum, średnią oraz wyliczony krok fazy. Pozwala to wychwycić nasycenie albo zły offset jeszcze przed zapisem danych do FPGA.

8.3 Walidacja połączenia

Przycisk *Check* generuje krótki skrypt odczytujący rejestr sterujący spod adresu `0x44A00000`. Jeżeli XSDB połączy się z serwerem `hw_server`, wybierze MicroBlaze i wykona `mrd`, można uznać, że ścieżka JTAG i mapa adresowa są dostępne. Dopiero po takim teście zaleca się przesyłanie całej fali.

Rozdział 9

Weryfikacja symulacyjna

9.1 Testbench

Testbench `tb_awg_core.sv` generuje zegar, reset i sygnały konfiguracyjne, a następnie zapisuje do CSV adres pamięci, próbkę surową, próbkę po skalowaniu i bit sigma-delta. Pierwszy fragment symulacji pracuje z pełnym wzmocnieniem, a w drugim wzmocnienie zostaje zmniejszone. Pozwala to sprawdzić zarówno adresowanie DDS, jak i reakcję potoku skalującego.

Rozdział 10

Synteza i implementacja

10.1 Wykorzystanie zasobów

Projekt został zaimplementowany w Vivado 2018.3 dla układu Zynq-7010. Raport po rozmieszczeniu wskazuje wykorzystanie przedstawione w tabeli 10.1. Wartości obejmują cały system z MicroBlaze, pamięcią programu, interkonektem AXI, debugiem i rdzeniem AWG, a nie wyłącznie sam generator.

Tabela 10.1: Wykorzystanie wybranych zasobów po rozmieszczeniu.

Zasób	Użyto	Dostępne	Wykorzystanie
Slice LUTs	1677	17600	9,53%
Slice Registers	1732	35200	4,92%
Block RAM Tile	18	60	30,00%
DSP48E1	1	80	1,25%
Bonded IOB	6	100	6,00%

Największy procentowo udział ma BRAM. Większość tych bloków jest związana z pamięcią lokalną MicroBlaze, a część z pamięcią przebiegu. Pojedynczy DSP jest wykorzystywany przez mnożenie w bloku skalowania. Zużycie LUT i rejestrów pozostawia duży zapas na rozwój projektu.

10.2 Analiza czasowa

Zegar generowany przez Clocking Wizard ma okres 10 ns, czyli częstotliwość 100 MHz. Po routingu uzyskano:

- WNS (Worst Negative Slack): 2,621 ns,
- TNS (Total Negative Slack): 0,000 ns,
- liczba punktów końcowych z naruszeniem setup: 0,

- WHS (Worst Hold Slack): 0,036 ns,
- liczba punktów końcowych z naruszeniem hold: 0.

Raport Vivado stwierdza, że wszystkie ograniczenia czasowe użytkownika są spełnione. Dodatni WNS oznacza, że najgorsza ścieżka setup kończy się ponad 2,6 ns przed wymaganym zboczem zegara. Uzyskany wynik potwierdza skuteczność potokowania bloku skalującego.

10.3 Ograniczenia XDC

Zewnętrzny zegar 125 MHz ma zdefiniowany okres 8 ns i jitter 0,08 ns. Wyjście sigma-delta nie jest próbkowane przez synchroniczne urządzenie zewnętrzne, dlatego zostało wyłączone z analizy zewnętrznego czasu wyjścia poleceniem `set_false_path`. Również przyciski resetu są asynchroniczne i nie powinny być traktowane jako zwykłe dane czasowe. Takie ograniczenia nie naprawiają logiki, ale poprawnie opisują rzeczywiste środowisko pracy układu.

Rozdział 11

Pomiary na oscyloskopie

11.1 Przebieg sinusoidalny

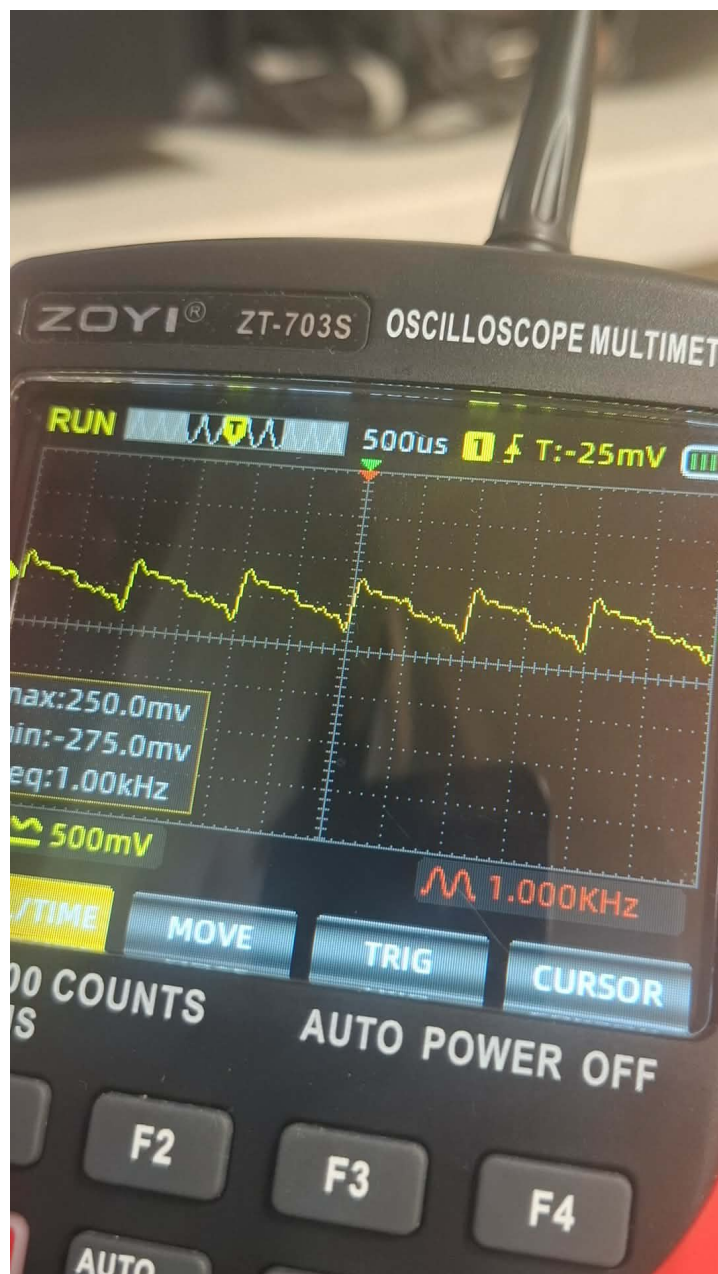
Po zaprogramowaniu FPGA i uruchomieniu generatora zmierzono przebieg o częstotliwości około 1 kHz. Na ekranie oscyloskopu widoczna jest wartość **Freq: 1.00 kHz**. Kształt jest zbliżony do sinusoidy, co potwierdza poprawne współdziałanie DDS, pamięci, skalowania i modulatora sigma–delta.



Rysunek 11.1: Przebieg sinusoidalny o częstotliwości około 1 kHz.

11.2 Przebieg niestandardowy

Drugie zdjęcie pokazuje przebieg o ostrzejszym kształcie i widocznych skokach. Jest to ważna demonstracja arbitralności generatora: pamięć nie musi zawierać sinusoidy, a tor wyjściowy odtwarza średnią odpowiadającą kolejno odczytywanym próbkom.



Rysunek 11.2: Przykład niestandardowego przebiegu uzyskanego z pamięci próbek.

11.3 Interpretacja wyników

Pomiary potwierdzają podstawową funkcjonalność, ale nie są pełną charakterystyką metrologiczną. Do oceny jakości generatora należałoby dodatkowo zmierzyć widmo, THD, poziom szumu, liniowość amplitudy, wpływ offsetu i pasmo filtra. Należy również oddzielić zniekształcenia pochodzące z tablicy próbek, modulatora, filtra i sondy pomiarowej.

Rozdział 12

Problemy napotkane podczas realizacji

12.1 Długa ścieżka kombinacyjna

Pierwszym istotnym problemem był niespełniony timing w torze od pamięci przez mnożnik i saturację do modulatora. Rozwiązaniem było dodanie rejestrów potokowych w `scaler.sv`. Zmiana zwiększyła opóźnienie, lecz poprawiła maksymalną częstotliwość pracy i doprowadziła do dodatniego WNS.

12.2 Sterowanie z komputera

Pierwotny kod SDK zawierał rozbudowaną obsługę UART. W praktyce działająca konfiguracja laboratoryjna korzystała z kabla Xilinx i XSDB. Utrzymywanie dwóch ścieżek zwiększało złożoność, a nie dawało korzyści w demonstracji. Usunięcie parsera UART z SDK zmniejszyło kod i jednoznacznie określiło wspierany sposób sterowania.

12.3 Aktualizacja całej tablicy

Przesłanie 4096 próbek jako osobnych zapisów `mwr` trwa zauważalnie dłużej niż pojedyncza zmiana konfiguracji. Z tego powodu aplikacja rozdziela operacje *Apply Config* i *Send Waveform*. Częstotliwość, gain i offset można zmieniać bez ponownego przesyłania próbek.

12.4 Charakter wyjścia sigma–delta

Bez właściwej filtracji na wyjściu widoczna jest składowa przełączająca i szum. Ostateczny kształt zależy od pasma oscyloskopu, sposobu podłączenia masy, wartości elementów RC oraz obciążenia. Jest to część toru analogowego, której nie można ocenić wyłącznie na podstawie symulacji RTL.

Rozdział 13

Możliwe kierunki rozwoju

13.1 Dedykowany tor DAC

Największą poprawę jakości dałoby zastosowanie zewnętrznego przetwornika DAC sterowanego równolegle lub przez szybki interfejs szeregowy. Pozwoliłoby to ograniczyć szum jednobitowej modulacji i zwiększyć użyteczne pasmo.

13.2 Szybsze ładowanie próbek

Zapis pojedynczych słów przez XSDB jest prosty, lecz nieoptymalny. Możliwe rozszerzenia obejmują:

- blokowy zapis pamięci przez XSDB,
- kontroler DMA,
- współdzieloną pamięć BRAM widoczną dla MicroBlaze,
- Ethernet lub USB implementowane jako docelowy interfejs użytkownika.

13.3 Dwa banki pamięci

Podwójne buforowanie pozwoliłoby ładować nowy przebieg do nieaktywnego banku, a następnie przełączyć bank na granicy okresu. Eliminowałoby to częściowo zapisane przebiegi i krótkie zakłócenia podczas aktualizacji.

13.4 Rozszerzenie funkcji generatora

Kolejne wersje mogłyby zawierać wiele kanałów, wyzwalanie, generację paczek, odtwarzanie sekwencji, modulację amplitudy i częstotliwości, zapis profili oraz automatyczny pomiar odpowiedzi badanego układu.

Rozdział 14

Podsumowanie

Zrealizowano kompletny generator przebiegów arbitralnych oparty na FPGA. Rdzeń sprzętowy łączy DDS, pamięć BRAM, potokowe skalowanie i modulator sigma-delta. Własny blok AXI4-Lite umożliwia zmianę częstotliwości, amplitudy, offsetu oraz zawartości pamięci. Aplikacja PyQt6 przygotowuje przebieg i steruje układem przez XSDB/JTAG.

Rezygnacja z nieużywanej obsługi UART w SDK uprościła firmware i usunęła niejednoznaczność dotyczącą wspieranego toru komunikacji. Minimalna aplikacja startowa kompiluje się poprawnie i ustawia domyślny przebieg 1 kHz. Dalsze zmiany są realizowane przez bezpośrednie zapisy rejestrów AXI.

Symulacja potwierdziła przepływ próbek i działanie skalera. Implementacja dla Zynq-7010 spełnia ograniczenia czasowe przy 100 MHz z WNS 2,621 ns. Pomiar na oscyloskopie potwierdził uzyskanie przebiegu 1 kHz oraz możliwość generowania niestandardowych kształtów. Projekt spełnił więc główny cel dydaktyczny: połączył projektowanie RTL, integrację systemu MicroBlaze/AXI, oprogramowanie narzędziowe i pomiar fizycznego sygnału.

Dodatek A

Aktualny minimalny firmware SDK

Listing A.1: Plik main.c.

```
1 #include "xparameters.h"
2 #include "xil_io.h"
3 #include "xil_printf.h"
4 #include <stdint.h>
5
6 #define AWG_BASEADDR XPAR_AWG_AXI_O_S00_AXI_BASEADDR
7 #define REG_CONTROL 0x00u
8 #define REG_PHASE_STEP 0x04u
9 #define REG_GAIN_OFFSET 0x08u
10 #define CTRL_ENABLE 0x00000001u
11 #define CTRL_RESET 0x00000002u
12 #define PHASE_STEP_1KHZ 0x0000A7C6u
13 #define GAIN_UNITY_Q16 0xFFFFu
14
15 static void awg_reset(void)
16 {
17     Xil_Out32(AWG_BASEADDR + REG_CONTROL, CTRL_RESET);
18     Xil_Out32(AWG_BASEADDR + REG_CONTROL, 0x00000000u);
19 }
20
21 static void awg_config(uint32_t phase_step,
22                        uint16_t gain_q16,
23                        uint16_t offset)
24 {
25     uint32_t gain_offset = ((uint32_t)offset << 16) | gain_q16;
26     Xil_Out32(AWG_BASEADDR + REG_PHASE_STEP, phase_step);
27     Xil_Out32(AWG_BASEADDR + REG_GAIN_OFFSET, gain_offset);
28 }
29
30 int main(void)
31 {
32     awg_reset();
33     awg_config(PHASE_STEP_1KHZ, GAIN_UNITY_Q16, 0u);
34     Xil_Out32(AWG_BASEADDR + REG_CONTROL, CTRL_ENABLE);
35 }
```

```
36     xil_printf("AWG initialized; runtime control: XSDB/JTAG.\r\n");
37     while (1) { }
38     return 0;
39 }
```

Dodatek B

Procedura uruchomienia

1. Otworzyć projekt `gen_arb_z_ip.xpr` w Vivado 2018.3.
2. Wygenerować bitstream i wyeksportować sprzęt do SDK.
3. Zbudować aplikację `awg_gen` dla MicroBlaze.
4. Zaprogramować FPGA i uruchomić aplikację startową.
5. Uruchomić `tools/awg_qt/awg_qt.py`.
6. Ustawić ścieżkę do `xsdb.bat` lub `xsct.bat`.
7. Użyć przycisku *Check*, aby odczytać rejestr sterujący.
8. Wybrać przebieg i parametry, a następnie użyć *Send Waveform*.
9. Podłączyć oscyloskop do Pmod JA1 i masy układu.
10. Ustawić podstawę czasu i wyzwalanie odpowiednie dla zadanej częstotliwości.

Bibliografia

- [1] AMD Xilinx, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, dokumentacja narzędzia Vivado.
- [2] AMD Xilinx, *MicroBlaze Processor Reference Guide*, dokumentacja procesora soft-core.
- [3] ARM, *AMBA AXI and ACE Protocol Specification*, opis protokołu AXI4-Lite.
- [4] Pliki projektu: `awg_core.sv`, `dds_addr.sv`, `wave_bram.sv`, `scaler.sv`, `sigma_delta_1st.sv`.
- [5] Pliki projektu: `tools/awg_qt/awg_qt.py` oraz `README.md`.
- [6] Raporty Vivado: `design_awg_wrapper_utilization_placed.rpt` oraz `design_awg_wrapper_timing_summary_routed.rpt`.